# Trabalho Pratico 1 - Organização de Computadores

**Ingrid Luiza Rodrigues Silva - 5938**
**Otávio Francisco Sabino Tavares - 5912**

[1] Universidade Federal de Viçosa - Campus Florestal

ingrid.r.silva@ufv.br, otavio.sabino@ufv.br

## 1. Introdução

Nosso trabalho consiste na criação de uma versão simplificada de um montador Risc-V capaz de ler um arquivo no formato .asm e gerar como saída as instruções correspondentes em código binário. O objetivo principal é compreender o funcionamento interno de um montador, bem como a estrutura das instruções da arquitetura RISC-V, reforçando os conceitos estudados na disciplina de Organização de Computadores.

## 2. Desenvolvimento

### 2.1. Leitura e Análise das Instruções

Inicialmente, o programa realiza a leitura do arquivo de entrada no formato .asm. Esse arquivo contém as instruções em linguagem assembly que devem ser convertidas para código binário. A leitura é feita linha por linha, permitindo que cada instrução seja processada individualmente.

Após a leitura, cada linha passa por uma etapa de análise, na qual a instrução é separada em seus componentes principais, como o nome da operação e seus operandos. Esse processo é essencial para identificar corretamente os registradores e valores imediatos envolvidos em cada instrução.

### 2.2. Classificação e Geração do Código Binário

Em seguida, o programa classifica cada instrução de acordo com seu tipo, como instruções do tipo R, I, S ou B. Essa classificação é importante, pois cada tipo possui um formato específico de organização dos bits, conforme definido na arquitetura RISC-V.

Para a geração do código binário, o programa organiza os campos de cada instrução de acordo com seu formato.

No caso das instruções do tipo R, os campos são organizados na ordem: funct7, rs2, rs1, funct3, rd e opcode, formando uma instrução final de 32 bits.

Para instruções do tipo I, como *addi* e *lw*, os campos seguem a organização: imediato, rs1, funct3, rd e opcode.

Já para instruções do tipo S, como *sw*, o valor imediato é dividido em duas partes e posicionado em regiões distintas da instrução.

No caso das instruções do tipo B, como *bne*, o valor de desvio também é dividido e reorganizado em posições não contíguas, conforme especificado pela arquitetura.

Os registradores, identificados de x0 a x31, são convertidos para valores binários de 5 bits. Para os valores imediatos, o programa realiza a conversão do valor decimal para binário, aplicando extensão de sinal quando necessário.

### 2.3. Resultado

Para garantir a organização e escalabilidade do montador, foi implementada a classe Comando. Esta estrutura funciona como um buffer de instrução, armazenando temporariamente todos os campos necessários (Opcode, Funct3, Funct7, registradores e imediato) antes da concatenação final. Essa abordagem facilita a manutenção do código, permitindo que diferentes formatos de instrução (R, I, S, B) sejam processados por uma interface comum.

```
class Comando:
    def __init__(self):
        self.tipo = ''
        self.funct7 = ''
        self.rs2 = ''
        self.rs1 = ''
        self.funct3 = ''
        self.rd = ''
        self.opcode = ''
        self.immediate = ''
```

**Figure 1. Classe Comando**

O trecho do código da Figura 2 é responsável por tratar os operandos das instruções. Inicialmente, remove-se o prefixo *x* dos registradores para permitir sua conversão em valores numéricos. Em seguida, de acordo com o tipo da instrução (R, I, S ou B), os campos *rd*, *rs1*, *rs2* e o imediato são convertidos para binário com tamanho fixo, garantindo que cada instrução seja montada corretamente conforme o padrão da arquitetura RISC-V.

```
for i in range (1, 4):
    instrucao[i].replace("x", "")


if c.tipo == 'r':
    c.rd  = format(int(instrucao[1]), "05b")
    c.rs1 = format(int(instrucao[2]), "05b")
    c.rs2 = format(int(instrucao[3]), "05b")

elif c.tipo == 'i':
    c.rd  = format(int(instrucao[1]), "05b")
    c.rs1 = format(int(instrucao[2]), "05b")
    c.immediate = binario(int(instrucao[3]), 12)

elif c.tipo == 's':
    c.rs2 = format(int(instrucao[1]), "05b")
    c.rs1 = format(int(instrucao[2]), "05b")
    c.immediate = format(int(instrucao[3]), '012b')

elif c.tipo == 'b':
    c.rs1 = format(int(instrucao[1]), "05b")
    c.rs2 = format(int(instrucao[2]), "05b")
    c.immediate = format(int(instrucao[3]), '013b')
```

**Figure 2. Tratamento dos Operandos**

```python
def indentificaComando(instrucao, c):

    match instrucao[0]:
        case 'add' | 'beq' | 'addi' | 'lb' | 'sb' | 'sub':
            c.funct3 = '000'

        case 'lh' | 'sh' | 'sll' | 'bne':
            c.funct3 = '001'

        case 'lw' | 'sw':
            c.funct3 = '010'

        case 'xor':
            c.funct3 = '100'

        case 'srl':
            c.funct3 = '101'

        case 'ori' | 'or':
            c.funct3 = '110'

        case 'andi' | 'and':
            c.funct3 = '111'
```

**Figure 3. Funct 3**

A Figura 3 apresenta o trecho responsável pela definição do campo *funct3*. Utilizando a estrutura *match-case*, o programa verifica o nome da instrução e atribui automaticamente o valor binário correspondente.

Por exemplo, instruções como *add*, *sub* e *addi* recebem *funct3 = 000*, enquanto instruções como *and* e *andi* recebem *funct3 = 111*. Essa associação segue o padrão definido pela arquitetura RISC-V, garantindo a correta identificação das operações.

De forma semelhante, o campo *funct7* também é definido por meio de estruturas condicionais, nas quais cada instrução recebe o valor binário correspondente. Esse campo é especialmente importante para diferenciar instruções que possuem o mesmo *funct3* e *opcode*, como é o caso de *add* e *sub*.

## 2.4. Testes Realizados

Para validar o funcionamento do montador, foram realizados testes utilizando diferentes conjuntos de instruções em assembly. Em cada caso, o programa foi capaz de converter corretamente as instruções para sua representação binária de 32 bits.

A Figura 4 apresenta um exemplo de entrada contendo instruções do tipo R, I, S e B, juntamente com a saída gerada pelo programa.

```
teste.asm
add x2, x0, x1
add x6, x0, x2
sub x3, x6, x2
xor x4, x2, x3
srl x0, x2, x2
sw x2, x1, 4
bne x1, x5, 40
addi x1, x1, 3
00000000000100000000000100110011
00000000001000000000001100110011
01000000001000110000000110110011
00000000001100010100001000110011
00000000001000010101000000110011
00100000001000001010000000100011
00101000010100001001000001100011
00000000001100001000000010010011
```

**Figure 4. Teste 1**

A Figura 5 apresenta um segundo conjunto de testes, contendo instruções do tipo R e I, com foco em operações lógicas e de deslocamento, como *sll*, *or* e *andi*.

```
teste2.asm
add x2, x0, x1
sll x1, x2, x2
or x2, x2, x1
andi x2, x1, 16
addi x3, x2, -243
00000000000100000000000100110011
00000000001000010001000010110011
00000000000100010110000100110011
00000001000000000111100010010011
11110000110100010000000110010011
```

**Figure 5. Teste 2**

Nesse teste, foi possível verificar a correta definição dos campos *funct3*, *funct7* e *opcode*, além da conversão adequada dos valores imediatos. Os resultados demonstram que o montador consegue tratar diferentes operações mantendo a estrutura binária exigida pela arquitetura RISC-V.

A Figura 6 apresenta um terceiro conjunto de testes com instruções do tipo R, incluindo operações aritméticas e lógicas como *add*, *sub*, *xor* e *srl*.

```
teste3.asm
add x2, x0, x1
add x6, x0, x2
sub x3, x6, x2
xor x4, x2, x3
srl x0, x2, x2
00000000000100000000000100110011
00000000001000000000001100110011
01000000001000110000000110110011
00000000001100010100001000110011
00000000001000010101000000110011
```

**Figure 6. Teste 3**

Esse teste permite verificar a correta diferenciação entre instruções que possuem o mesmo *opcode*, mas se distinguem pelos campos *funct3* e *funct7*, como é o caso de *add* e *sub*. Os resultados confirmam que o montador realiza essa distinção corretamente na geração do código binário.

Observa-se que cada instrução de entrada foi corretamente interpretada, com seus campos organizados de acordo com o formato especificado pela arquitetura RISC-V.

Testes adicionais foram realizados para garantir a consistência da conversão, incluindo diferentes valores de registradores e imediatos, demonstrando a robustez do montador desenvolvido.

### 2.5. Requisitos e Configuração do Ambiente

O repositório do projeto se encontra no GitHub[1].

O montador foi desenvolvido utilizando a linguagem Python. Para a execução em ambiente Linux, é necessário que o interpretador Python esteja instalado. Caso não esteja, ele pode ser instalado via terminal com os comandos sudo apt update e sudo apt install python3.

Não foram utilizadas bibliotecas externas, sendo necessário apenas o interpretador padrão para o funcionamento do código.

O programa suporta as duas formas de saída exigidas na especificação. A execução deve seguir os formatos abaixo:

### 2.6. Instruções de Execução

O montador oferece duas formas de exibição dos resultados, conforme as especificações do projeto. Para a **Exibição no Terminal**, caso o usuário passe apenas o arquivo de entrada via linha de comando, o programa exibe todas as instruções convertidas para binário diretamente no console (Exemplo: *python3 src/montador.py entrada.asm*, considerando que se esteja na pasta do projeto, "*C:/[...]/tp1oc*").

Já para o **Salvamento em Arquivo**, o usuário deve adicionar o nome de um arquivo, e o programa grava o código binário gerado no arquivo especificado (Exemplo: *python3 src/montador.py entrada.asm saida.txt*).

[1] https://github.com/tavinEscada/tp1oc

É possível, ainda, executar usando o **Makefile**, com comandos do tipo *make run ENTRADA=teste.asm SAIDA=saida.txt* para saída com arquivo, ou do tipo *make run ENTRADA=teste.asm* para saída no terminal.

## 3. Conclusão

O desenvolvimento deste montador em Python permitiu a compreensão prática da tradução de mnemônicos para linguagem de máquina. A estrutura modular do código facilitou a implementação das instruções, garantindo que os formatos R, I, S e B fossem gerados corretamente. O resultado final atende às especificações de entrada (.asm) e saída (terminal ou arquivo), consolidando os conhecimentos de organização de computadores e arquitetura RISC-V.

## 4. Referências


RISC-V Foundation. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Disponível em: https://riscv.org

Patterson, David A.; Hennessy, John L. *Computer Organization and Design RISC-V Edition*. Morgan Kaufmann, 2017.

Python Software Foundation. *Python Documentation*. Disponível em: https://docs.python.org

GeeksforGeeks. *Binary Representation in Python*. Disponível em: https://www.geeksforgeeks.org

GOMES, Ana Maria. *Lendo e Escrevendo Arquivos de Texto em Python*. Asimov Academy. Disponível em: https://hub.asimov.academy/tutorial/lendo-e-escrevendo-arquivos-de-texto-em-python/